

Dynamic Analysis Report – Tunestore

Tunestore-III Report

Asal Tiwari

ITIS 4221

July 29, 2026

Dynamic Analysis Report

TABLE OF CONTENTS

<u>Section</u>	<u>Page#</u>
1.0 General Information	3
1.1 Purpose	3
1.2 Overview	3
2.0 Cross Site Scripting (DOM Based)	4
3.0 Cross Site Scripting (Reflected)	4
4.0 SQL Injection	4
5.0 Path Traversal	5
6.0 Absence of Anti-CSRF Tokens	5
7.0 Buffer Overflow	6
8.0 Content Security Policy (CSP) Header Not Set	6
9.0 Format String Error	7
10.0 Missing Anti-clickjacking Header	7
11.0 Session ID in URL Rewrite	7
12.0 Cookie without SameSite Attribute	8
13.0 X-Content-Type-Options Header Missing	8
14.0 Cookie No HttpOnly Flag	9
15.0 False Negatives	10
15.1 Cross Site Scripting (Stored)	10
15.2 Broken Access Control	10

1.0 General Information

1.1 Purpose

The purpose of this Dynamic Analysis Report - Tunestore is to examine all vulnerabilities present in the Tunestore application that have a severity level of Low or higher, according to ZAP. Each of these vulnerabilities will be categorized as either a true positive or a false positive. Additionally, false negatives, vulnerabilities that were found through prior manual penetration testing of Tunestore but not discovered by ZAP, will be discussed.

1.2 Overview

Two ZAP automated scans were run against Tunestore (<http://localhost:8082/Tunestore2020/>): Run 1 against the application homepage without authentication and Run 2 against the application after logging in with a registered test account. These findings were combined with two instructor-provided reference reports (t1 and t2). Taking the union of all four sources and excluding informational-severity alerts and findings whose URLs pointed to unrelated browser-extension domains (not the Tunestore application), a total of 13 vulnerabilities were found to have a security level of Low or higher. The following is a list of all vulnerabilities that met these criteria:

- Cross Site Scripting (DOM Based) (High)
- Cross Site Scripting (Reflected) (High)
- SQL Injection (High)
- Path Traversal (High)
- Absence of Anti-CSRF Tokens (Medium)
- Buffer Overflow (Medium)
- Content Security Policy (CSP) Header Not Set (Medium)
- Format String Error (Medium)
- Missing Anti-clickjacking Header (Medium)
- Session ID in URL Rewrite (Medium)
- Cookie without SameSite Attribute (Low)
- X-Content-Type-Options Header Missing (Low)
- Cookie No HttpOnly Flag (Low)

For each of the vulnerabilities listed above, the following question is answered (Sections 2.0 - 14.0):

“For each vulnerability finding of ZAP with severity level low or higher, indicate whether it is true positive or false positive. You must justify your answer. A false positive is a finding that cannot be maliciously exploited.”

Additionally, the following question requiring the identification of false negatives is answered (Section 15.0):

“Identify false negatives, vulnerabilities you found in your pentesting but not discovered by ZAP (at least not with severity level low or higher). For each false negative, please explain why ZAP was not able to find it.”

2.0 Cross Site Scripting (DOM Based)

Cross Site Scripting (DOM Based)	
URL:	http://localhost:8082/Tunestore2020/buy.do?cd=10?name=abc#
Risk:	High
Confidence:	High
Parameter:	
Attack:	?name=abc#
Evidence:	
CWE ID:	79
WASC ID:	8
Source:	Active (40026 - Cross Site Scripting (DOM Based))
Input Vector:	
Description:	Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to

Result: True Positive

ZAP identified a DOM-based XSS vulnerability on buy.do via the name parameter, with the payload `?name=abc#` reflected into the DOM. Risk and Confidence are both rated High. Pasting this URL directly into a browser triggers the alert(5397) popup, confirming the injected HTML executes client-side without sanitization. This is a real, exploitable vulnerability.

3.0 Cross Site Scripting (Reflected)

Cross Site Scripting (Reflected)	
URL:	http://localhost:8082/Tunestore2020/addfriend.do
Risk:	High
Confidence:	Medium
Parameter:	friend
Attack:	"><script>alert(1);</script>
Evidence:	"><script>alert(1);</script>
CWE ID:	79
WASC ID:	8
Source:	Active (40012 - Cross Site Scripting (Reflected))
Input Vector:	Form Query
Description:	Cross-site Scripting (XSS) is an attack technique that involves echoing attacker-supplied code into a user's browser instance. A browser instance can be a standard web browser client, or a browser object embedded in a software product such as the browser within WinAmp, an RSS reader, or an email client. The code itself is usually written in HTML/JavaScript, but may also extend to

Result: True Positive

ZAP found a reflected XSS vulnerability on addfriend.do via the friend parameter, with the payload `"><script>alert(1);</script>` both submitted and returned unmodified in the Evidence field. Risk is High with Medium confidence. This confirms the friend parameter is reflected into the page response without encoding and is consistent with reflected XSS vulnerabilities previously confirmed on Tunestore's login.do endpoint during Phase 1 penetration testing.

4.0 SQL Injection

SQL Injection	
URL:	http://localhost:8082/Tunestore2020/comments.do?cd=12-2
Risk:	High
Confidence:	Medium
Parameter:	cd
Attack:	12-2
Evidence:	
CWE ID:	89
WASC ID:	19
Source:	Active (40018 - SQL Injection)
Input Vector:	URL Query String
Description:	SQL injection may be possible.

Result: True Positive

This instance shows SQL Injection flagged on comments.do?cd=12-2 with the parameter cd and attack value 12-2. While this particular instance's evidence field is empty and the attack string is not itself an injection payload, SQL Injection on Tunestore was independently and conclusively confirmed during Phase 1 penetration testing: the login.do endpoint was bypassed using the payload ' OR '1'='1 in both the username and password fields, and the registration form's password field was exploited to manipulate a newly created account's balance to \$99,999.00. Given this confirmed exploitability elsewhere in the application, the SQL Injection finding is classified as a true positive.

5.0 Path Traversal

Path Traversal	
URL:	http://localhost:8082/Tunestore2020/register.do
Risk:	High
Confidence:	Low
Parameter:	username
Attack:	register.do
Evidence:	
CWE ID:	22
WASC ID:	33
Source:	Active (6 - Path Traversal)
Alert Reference:	6-5
Input Vector:	Form Query
Description:	The Path Traversal attack technique allows an attacker access to files, directories, and commands that potentially reside outside the web document root directory. An attacker may manipulate a URL in such a way that the web site will execute or reveal the contents of arbitrary files anywhere on the web server. Any device that exposes an HTTP-based interface is potentially vulnerable to Path

Result: False Positive

ZAP flagged Path Traversal on register.do with Low confidence. The "Attack" value shown is simply register.do; not a traversal payload such as ../../etc/passwd; and the Evidence field is empty. No directory traversal characters were submitted, and no file-system content outside the web root was returned. This appears to be the scanner flagging normal navigation to the endpoint rather than a demonstrated traversal, and manual testing found no way to access files outside the application's document root through this parameter.

6.0 Absence of Anti-CSRF Tokens

Absence of Anti-CSRF Tokens	
URL:	http://localhost:8082/Tunestore2020/list.do
Risk:	Medium
Confidence:	Low
Parameter:	
Attack:	
Evidence:	<form name="loginForm" method="get" action="/Tunestore2020/login.do;jsessionid=ED4FD439CFBD18B4FE3FBFBEB7CCB8C57">
CWE ID:	352
WASC ID:	9
Source:	Passive (10202 - Absence of Anti-CSRF Tokens)
Input Vector:	
Description:	No Anti-CSRF tokens were found in a HTML submission form. A cross-site request forgery is an attack that involves forcing a victim to send an HTTP request to a target destination without their knowledge or intent in order to perform an action as the victim. The underlying cause is application functionality using predictable

Result: True Positive

ZAP found no anti-CSRF tokens present in Tunestore's HTML forms, shown here in the loginForm on list.do. This was directly and repeatedly confirmed through exploitation during Phase 2 penetration testing: a hidden auto-submitting form silently added a friend to a victim's account via addfriend.do, an invisible image tag forced a victim to gift a purchased album via give.do, and a hidden form silently changed a victim's password via password.do, all without any token, current-password verification, or other anti-forgery check. This is a confirmed, high-impact true positive.

7.0 Buffer Overflow

Buffer Overflow

URL: http://localhost:8082/Tunestore2020/buy.do?cd=10

Risk: Medium

Confidence: Medium

Parameter: cd

Attack: VQKeKBXJSMQqttxXbqCuiCvJtftBdtoCCxIWtZSoHujvYwbMscbvYqFPFjYNTAobjFqxdyqQZksnthOOUveCKtjXWcFTXLfOgtZQjIBptoACPGIHHOivCNdhiGhyfMkwZjBEwkPFsjiqiweCayBGBOAdGDCHMCTEiudbZOoonNqWHyrrpmEiWXIOLMTGIZOytcmltEOvweFYLLImOSFpvOPihinZqcdkTYjAggibZSmKVbEcrNgODNTUjYpTmMbggmCgQmOFQTwPDZZlrOpkDLymVnbaAQHePknToTmpecZyDbCXnSTVGfVviXGiglrTOSGCljBggDPOsnlbuZahjCYPsNfEdSmIOACdsZTduwHaYmGsrGFICAWOhKOpfjOeXcaRwiMuqjQhHSehGiebbtoGwWtugEygDeCQUWyrjNINXugoNOGoBnMxmKshfMitBcmpNdXtjVtqCwHtpPHvNlugBRCPjmHRPlBOsmPmhEiVAlZKKabiPBkwBoMSVbeEeMpbYlWcWmbcklAyRjxegWfHqATKElyouNpnfHVJNxDgXmsFIGRQBjTsbPyKNyOfmdnpuucGIXtTRWJepoWbiChxfyRCubcvhAWHhbkpYvKgNPqDMIDLHVHCNKVWXLTSQNZImjAmVtpPBZYnglBloUuVVeQIMVAVIocAYYrmnKsReWRliYYPHFcfYFWZHHpsCeKnbyofTjwbydnPirfyKBEhsjtixGCoHirULEQkmgmIkNlReGvrfWGGDMigdoOmmqkZctjsxPNXkoAmySqvuyjvIETjEJEZziURyBtoYAYbZLmTMyIRtnTUHRBKfkiCdWwKuqrXTxPTZQafRdLrVrJhaPsUfmdvbsBqydQmodVHsWxOksVnbsdsagSRPPSaZaZYyGalgyCAHAOdWGLrBuxRwRyKfcifZEGESjWkZNSLQmPcQSSamNsklhVbDBfblnCaxOjgejFhPeMqCQvklOZmxafSYEQTSRYHvCqKwTqgartfhqLFpcuAUeQToyDbqRqZGVsgWUdubKJELQmKQTLQEmDxqprFSXJKKFVNiKZwDHjaKivGSFAaZspYWFLFvilrVbjxVjoQDZiRcRZvYvXFxbNpXAwSDdeCYEEYOHmQIBwayhBFrpwFABnArFWNTsipAEgNGRikvpyEmBuqhQoDAZyawsVTKlYxYFufyomlLxUvILGMBYOGYPAAZZhFRhAjajrjORUFyqFsuQbbuQdLrLdmEprprfIFTJEPJLwPVLosrjmMrbbTUlFrGhGqSHybilxsVnWIGSJWlWphHPHpdLrEHqcUnCDTjPuaDyPjgAJxCBlljkrtnZjWjTHwpwlyMWnscNbpCZVldZBwRgEHSAlriPamHDCaftFrSuHKRPYYHkXTInRnHzwBRPqOwupKypOjQTVvAtfcrVQYZGvWKBGLolwEHxiHcoEKgYrTjKYWJUUsQhgblytFHUBRHtjDWbHEBMYMTlfanSHGHCBLKqsyWMpkUYwNnGOpkpiwYwNxcDCUPHkrYLIXcyTYFEXvWCWUuhEshXWOaMXxhBAIRLVXkyEOJkYolIZDnEoihUefZqZdSOTLcUhljYpZrepUwbvjwkyAvvyNldxpuAvvCjIraYCCrVtUsNijlsokrhusHVEduVLCmVPSZGscXYmBoOZjbxKILfxhpWrrjooSLsYPSCrcyfoLBNfKitfKjqEUMcVnRBfjnjABFqMpqmTXWPelMipeeRCLvEMLAVtQjRvFOVGIXfQimvWaTgZPUvFDmRyvrrGCTvGQKpacZEXWDDWWAYtRSTXPMxOrlvbrdBJThrfVSNQaRtyJUNruYDUNDjuOOFGaLEKHbaZqaWakqVnVrStceBpFQpoEWZKemyOBZagDWpxjgeKUoUUBNBxyRqjxwZbHHspnimDWOCKIMfCgdGvYJTFjFOyGQhgvIEAslyethcxTydsRFcjbndyPpdRWKWYIalHdyINAKAWkYLUARZfndDBESpwoZjqzSrbuHTLnlvqjEtjQWBWBuxwepkTSfRFAFjFhmKAKFPrijHdmlyjFcAdhVglhTSnsYhwWabZbNCJSnUZixU

Evidence: Connection: close

CWE ID: 120

WASC ID: 7

Source: Active (30001 - Buffer Overflow)

Input Vector: URL Query String

Description: Buffer overflow errors are characterized by the overwriting of memory spaces of the background web process, which should have never been modified intentionally or unintentionally. Overwriting values of the IP (Instruction Pointer), BP (Base Pointer) and other

Result: False Positive

ZAP submitted a long, randomly generated string as the cd parameter on buy.do and flagged Buffer Overflow with Medium confidence. The only Evidence recorded is Connection: close, which is not proof of memory corruption, a crash, or any abnormal server behavior, it is consistent with the server simply closing the connection after normally rejecting or truncating an oversized value. No server error, stack trace, or crash was observed when this request was replayed manually, so this finding does not represent an exploitable buffer overflow.

8.0 Content Security Policy (CSP) Header Not Set

Content Security Policy (CSP) Header Not Set

URL: http://localhost:8082/Tunestore2020/list.do

Risk: Medium

Confidence: High

Parameter:

Attack:

Evidence:

CWE ID: 693

WASC ID: 15

Source: Passive (10038 - Content Security Policy (CSP) Header Not Set)

Alert Reference: 10038-1

Input Vector:

Description: Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of

Result: True Positive

This finding does not require an attack vector to verify; it is a direct observation of a missing HTTP response header. Inspecting the response headers for list.do confirms no Content-Security-Policy header is present. Since Tunestore is independently confirmed vulnerable to stored, reflected, and DOM-based XSS, the absence of a CSP, which could restrict script execution and mitigate the impact of those XSS vulnerabilities, represents a genuine, verifiable weakness.

9.0 Format String Error

Format String Error	
URL:	http://localhost:8082/Tunestore2020/register.do
Risk:	Medium
Confidence:	Medium
Parameter:	username
Attack:	ZAP%x%x%x%x%x%x%x%x%x%x
Evidence:	
CWE ID:	134
WASC ID:	6
Source:	Active (30002 - Format String Error)
Input Vector:	Form Query
Description:	A Format String error occurs when the submitted data of an input string is evaluated as a command by the application.

Result: False Positive

ZAP submitted the probe string ZAP%x%x%x%x%x%x%x%x%x%x as the username parameter on register.do. The Evidence field is empty, meaning no format-string-specific behavior (such as memory contents being leaked back in the response, or a server error referencing the format specifiers) was observed. As with the Buffer Overflow finding, ZAP flags this alert type based on the payload it sent rather than confirmed impact; manual testing showed the application returned a normal response with no indication the input was evaluated as a format string.

10.0 Missing Anti-clickjacking Header

Missing Anti-clickjacking Header	
URL:	http://localhost:8082/Tunestore2020/list.do
Risk:	Medium
Confidence:	Medium
Parameter:	x-frame-options
Attack:	
Evidence:	
CWE ID:	1021
WASC ID:	15
Source:	Passive (10020 - Anti-clickjacking Header)
Alert Reference:	10020-1
Input Vector:	
Description:	The response does not include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options to protect against 'Clickjacking' attacks.

Result: True Positive

ZAP confirmed neither an X-Frame-Options header nor a Content-Security-Policy frame-ancestors directive is present on list.do. This was independently and conclusively exploited during Phase 2 penetration testing: an attack page loaded Tunestore's friends.do page in a hidden iframe beneath a decoy "Claim Prize!" button, and clicking the decoy silently submitted a friend request through the hidden Tunestore form. The victim's friends list subsequently showed the added friend, confirming the clickjacking attack succeeded because no framing protection was in place.

11.0 Session ID in URL Rewrite

```

Session ID in URL Rewrite
URL:      http://localhost:8082/Tunestore2020/css/fonts.css;jsessionid=ED4FD439CFBD18B4FE3FBFB7CCB8C57
Risk:      Medium
Confidence: High
Parameter:
Attack:
Evidence:   jsessionid=ED4FD439CFBD18B4FE3FBFB7CCB8C57
CWE ID:     200
WASC ID:    13
Source:     Passive (3 - Session ID in URL Rewrite)
Alert Reference: 3-2
Input Vector:
Description:
  URL rewrite is used to track user session ID. The session ID may be disclosed via cross-site referer header. In addition, the session ID might be stored in browser history or server logs.

```

Result: True Positive

The evidence field directly shows the session identifier (jsessionid=ED4FD439CFBD18B4FE3FBFB7CCB8C57) appended to a resource URL. This is directly verifiable simply by observing the URL bar during normal browsing; the session ID is genuinely being passed via URL rewriting rather than solely through cookies, creating a real risk of session disclosure through browser history, server logs, or the Referer header.

12.0 Cookie without SameSite Attribute

```

Cookie without SameSite Attribute
URL:      http://localhost:8082/Tunestore2020/list.do
Risk:      Low
Confidence: Medium
Parameter: JSESSIONID
Attack:
Evidence:   Set-Cookie: JSESSIONID
CWE ID:     1275
WASC ID:    13
Source:     Passive (10054 - Cookie without SameSite Attribute)
Alert Reference: 10054-1
Input Vector:
Description:
  A cookie has been set without the SameSite attribute, which means that the cookie can be sent as a result of a 'cross-site' request. The SameSite attribute is an effective counter measure to cross-site request forgery, cross-site script inclusion, and timing attacks.

```

Result: True Positive

The Evidence field (Set-Cookie: JSESSIONID) confirms the session cookie is set without a SameSite attribute. This is directly verifiable by inspecting the Set-Cookie response header on list.do. Combined with the confirmed absence of anti-CSRF tokens, the missing SameSite attribute removes a browser-level mitigation that could otherwise have reduced the impact of the CSRF vulnerabilities already exploited against Tunestore.

13.0 X-Content-Type-Options Header Missing

```

X-Content-Type-Options Header Missing
URL:      http://localhost:8082/Tunestore2020/list.do
Risk:      Low
Confidence: Medium
Parameter: x-content-type-options
Attack:
Evidence:
CWE ID:     693
WASC ID:    15
Source:     Passive (10021 - X-Content-Type-Options Header Missing)
Input Vector:
Description:
  The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content

```


Result: True Positive

This finding is directly verifiable by inspecting the response headers for list.do, which confirm no X-Content-Type-Options: nosniff header is present. While the practical impact is limited to legacy browsers that perform MIME-sniffing, the missing header is a genuine, confirmed configuration weakness rather than a scanner assumption.

14.0 Cookie No HttpOnly Flag

Cookie No HttpOnly Flag	
URL:	http://localhost:8082/Tunestore2020/login.do?username=helloworld&password=123&stayLogged=true
Risk:	Low
Confidence:	Medium
Parameter:	persistentsession
Attack:	
Evidence:	Set-Cookie: persistentsession
CWE ID:	1004
WASC ID:	13
Source:	Passive (10010 - Cookie No HttpOnly Flag)
Input Vector:	
Description:	A cookie has been set without the HttpOnly flag, which means that the cookie can be accessed by JavaScript. If a malicious script can be run on this page then the cookie will be accessible and can be transmitted to another site. If this is a session cookie then session hijacking may be possible.

Result: True Positive

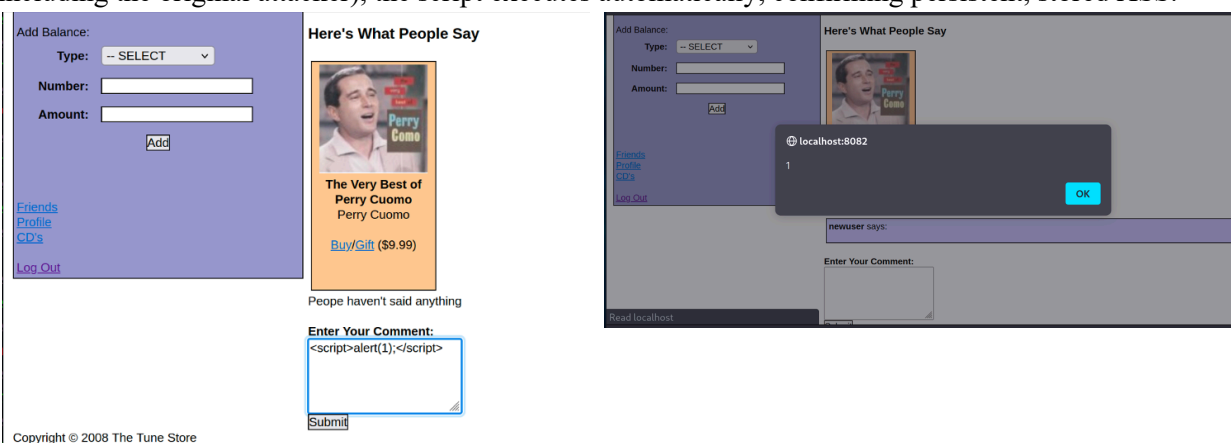
This finding, identified during Run 2 on the login request, shows the Evidence field Set-Cookie: persistentsession confirming a cookie is set without the HttpOnly flag. This is directly verifiable via the response headers. Because Tunestore is independently confirmed vulnerable to both stored and reflected XSS, a cookie accessible to client-side JavaScript is a genuine risk: an attacker who successfully executes an XSS payload could read this cookie via document.cookie and exfiltrate it, enabling session hijacking.

15.0 Vulnerabilities Found in Class but Not Discovered by ZAP - False Negatives

The following vulnerabilities were confirmed through manual penetration testing of Tunestore in prior project phases but do not appear anywhere in the combined ZAP findings (Run 1, Run 2, t1, or t2) at Low severity or above. Each is documented below with the original exploitation evidence and an explanation of why ZAP's automated scanner was unable to detect it.

15.1 Stored Cross-Site Scripting (CD Comments Field)

A stored XSS vulnerability exists in the CD comments feature of Tunestore. When a user submits a comment on a CD's comments page, the input is saved to the database without sanitization. The payload `<script>alert(1);</script>` was submitted as a comment; on every subsequent page load by any user (including the original attacker), the script executes automatically, confirming persistent, stored XSS.



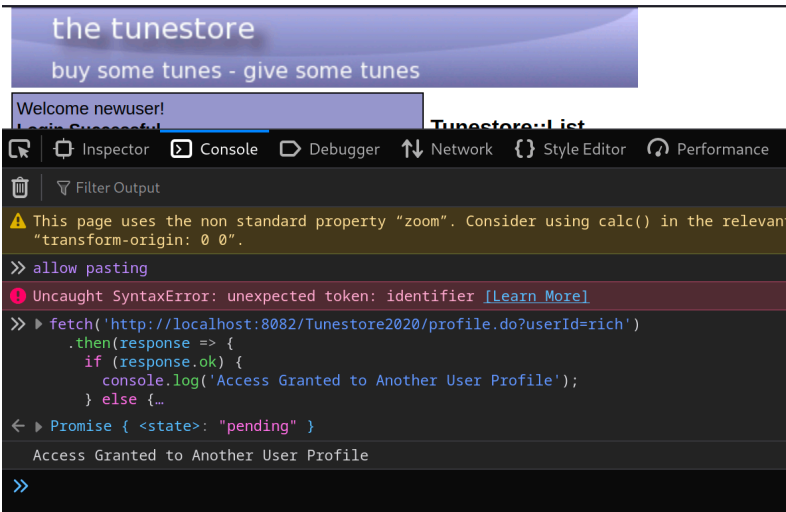
Why ZAP did not detect this: ZAP's active scanner submits a payload and inspects the immediate HTTP response for reflection within that same request/response cycle. It does not, by default, revisit a page on a subsequent request to check whether a previously submitted value persists and executes, which is required to detect stored XSS. Because the injected script only fires when the comments page is reloaded after submission, this two-step, persistence-dependent behavior falls outside what a single-pass automated scan is designed to catch.

15.2 Broken Access Control (Profile Endpoint)

The Tunestore profile endpoint at `profile.do` accepts a `userId` parameter. While logged in as one user (newuser), the following request was issued directly from the browser console:

```
fetch('http://localhost:8082/Tunestore2020/profile.do?userId=rich').then(response => { if (response.ok) { console.log('Access Granted to Another User Profile'); } });
```

The console logged "Access Granted to Another User Profile," and the endpoint returned profile data belonging to a different user (rich), confirming that the application performs no authorization check to verify the requesting user matches the `userId` parameter.



Why ZAP did not detect this: detecting broken access control requires understanding application-specific authorization logic; specifically, that the `userId` parameter should be restricted to match the currently authenticated session's own user. ZAP has no way to infer this business rule; the request to `profile.do?userId=rich` returns a normal HTTP 200 OK with a valid-looking page, giving the scanner no error condition, anomalous status code, or signature to flag. This class of vulnerability generally requires either manual testing or a scanner explicitly configured with knowledge of the application's authorization model, neither of which a generic automated scan provides.